

**** LAB ** Strutturare Vagrantfile e playbook [21 aprile]**

Sito: [Virtuale](#)

Corso: Amministrazione di Sistemi T - Laboratorio di
Amministrazione di Sistemi T

Libro: ** LAB ** Strutturare Vagrantfile e playbook [21 aprile]

Stampato da: Lorenzo Bernardini

Data: venerdì, 24 aprile 2026, 11:54

Descrizione

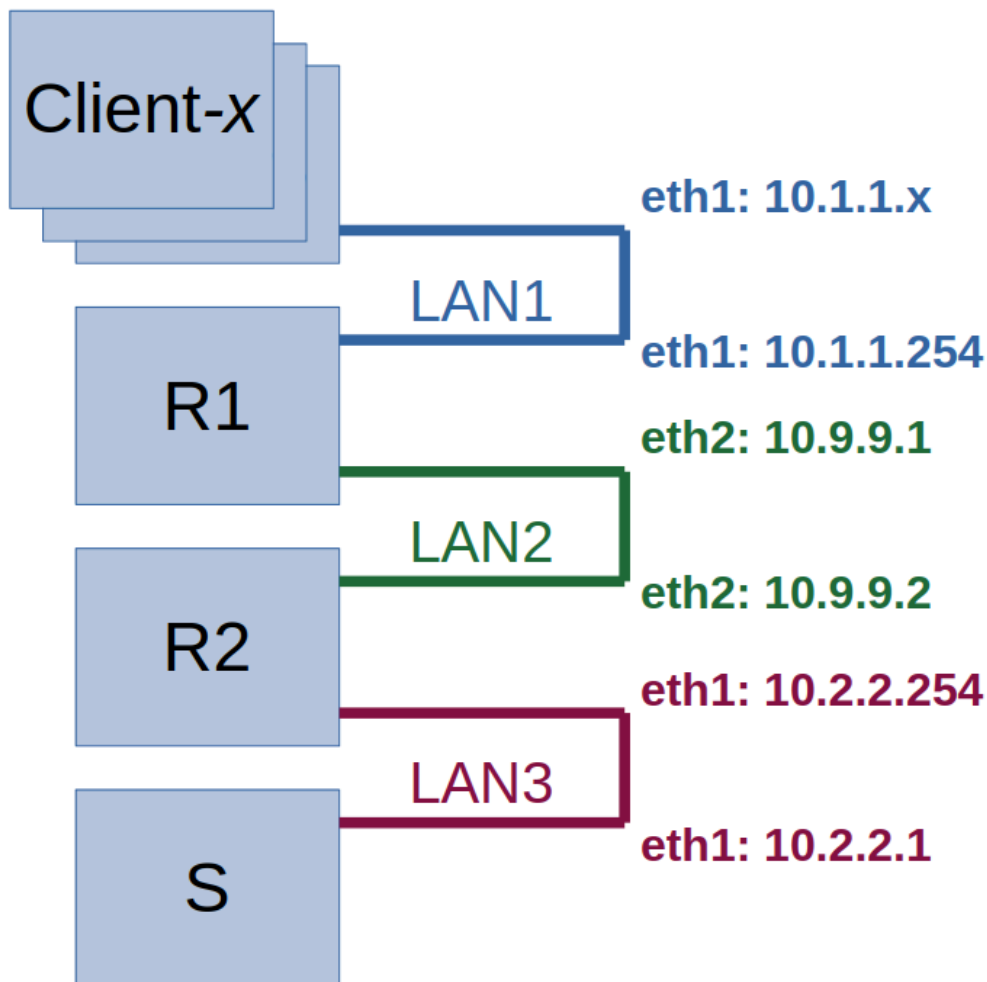
Attraverso qualche funzione avanzata di Vagrant e una struttura predefinita delle informazioni di Ansible, è possibile gestire più efficientemente scenari complessi riusando il codice di configurazione.

Sommario

1. **Caso di studio: multihop**
2. **Vagrant - un esempio di loop**
3. **Ansible - i ruoli**
4. **Ansible - variabili e cicli**
5. **Mettere tutto insieme**

1. Caso di studio: multihop

Facciamo riferimento alla topologia già vista, estesa al caso di n client configurati dal DHCP che gira su R1



Come possiamo semplificarne la realizzazione?

- Vagrant potrebbe automatizzare la creazione di più client?
- Ansible potrebbe curare tutti gli aspetti di installazione e configurazione?
- Si possono mettere a fattor comune i task Ansible identici in più VM?

2. Vagrant - un esempio di loop

Sappiamo già che in un singolo Vagrantfile possiamo definire più VM.

Se queste hanno gli stessi parametri si può utilizzare un [loop Ruby](#) per sintetizzarne la definizione, ad esempio:

```
(1..3).each do |i|  
  config.vm.define "client-#{i}" do |machine|  
    machine.vm.hostname = "client-#{i}"  
    machine.vm.network "private_network", virtualbox____intnet: "LAN1", auto_config: false  
    machine.vm.synced_folder ".", "/vagrant", disabled: true  
    machine.vm.provision "ansible" do |ansible|  
      ansible.playbook = "site.yml"  
    end  
  end  
end
```

Nota: la riga in blu serve a non montare il folder condiviso tra VM e host, per velocizzare il boot e preservare un miglior isolamento

3. Ansible - i ruoli

Come accennato nell'[introduzione ad Ansible](#) è possibile organizzare i task in base ai ruoli da assegnare alla macchine da "provisionare".

Utilizzando le collocazioni di default previste da Ansible, si semplifica molto anche la sintassi dei playbook.

```
├─ roles
│   └─ role1
│       ├── files
│       │   ├── file-utile-1
│       │   └─ file-utile-1
│       ├── handlers
│       │   └─ main.yml
│       ├── tasks
│       │   └─ main.yml
│       └─ vars
│           └─ main.yml
│   └─ (altri ruoli)
├─ site.yml
└─ Vagrantfile
```

Con questa struttura, se in **site.yml** si scrive:

```
---
- name: "Provisioning clients"
  hosts: client-*
  become: true
  roles:
    - role1
```

per ogni host con nome compatibile con **client-***

- verranno eseguiti tutti i task presenti in `roles/role1/tasks/main.yml`
- verranno attivati se necessario tutti gli handler presenti in `roles/role1/handlers/main.yml`
- tutti i file presenti in `roles/role1/files` potranno essere referenziati col nome relativo a quel punto del filesystem
- si potranno definire variabili condivise tra task, in `roles/role1/vars/main.yml`

Inoltre la sintassi dei singoli **main.yml** è semplificata perchè la posizione implica l'intestazione (task, handler, ...). Ad esempio, in `roles/role1/tasks/main.yml` si potrà scrivere direttamente

```
- name: "Copia un file utile"
  ansible.builtin.copy:
    src: file-utile-1
    dest: /tmp/il-mio-file-utile
```

(notate anche il riferimento al file col semplice nome, visto che si trova nella cartella files)

4. Ansible - variabili e cicli

Esempio: si vogliono fare più modifiche allo stesso file, verificando che siano presenti diverse linee.

```
- name: Configurazione ~/.bashrc
  become: true
  ansible.builtin.lineinfile:
    path: '/home/vagrant/.bashrc'
    state: present
    line: "{{ item }}"
  loop:
    - 'export HTTP_PROXY="{{ proxy_url }}"'
    - 'export HTTPS_PROXY="{{ proxy_url }}"'
```

Il modulo **lineinfile** cerca righe esattamente contenenti la stringa specificata da **line**: e modifica il file per garantire che sia **present** o **absent** come specificato da **state**:

La variabile speciale **item** verrà sostituita ad ogni iterazione dagli elementi specificati come parametri di **loop**

Tutto ciò che è racchiuso da **{{ }}** è una variabile: la variabile **proxy_url** deve essere definita da qualche parte, il posto predefinito è **vars/main.yml** nella cartella del ruolo:

```
---
lab_num: 3
proxy_url: 'http://192.168.12{{ lab_num }}.249:8080'
```

5. Mettere tutto insieme

Utilizzando le indicazioni fornite, realizzare la topologia proposta all'inizio, installando automaticamente dnsmasq su R1, e configurando tutto ciò che è necessario.

Si sfruttino i ruoli per mettere a fattor comune le attività di base: correzione di allow-hotplug eth0, configurazione dei proxy se ci si trova in laboratorio.

Si definiscano ruoli specifici per la configurazione differenziata delle singole macchine.

[Soluzione](#) base

[Soluzione](#) corretta sintassi 2025 e R2 che eroga indirizzo statico a Server